

第 6 章：DQN (Deep Q-Network, 深度 Q 网络) 与经典改进

6.1 本章符号说明

符号/缩写	English	中文	含义
DQN	Deep Q-Network	深度 Q 网络	用神经网络近似动作价值函数 $Q(s,a)$ 。
(s,a,r,s',d)	Transition Tuple	转移样本五元组	当前状态、执行动作、即时奖励、下一状态和终止指示量。
d / done	Terminal Indicator	终止指示量	$d=1$ 表示 transition 后任务终止, 没有后续价值; $d=0$ 表示可以继续估计未来价值。代码常把这个布尔字段命名为 <code>done</code> 。
$\mathcal{A}$ / $ \mathcal{A} $	Action Set / Number of Actions	动作集合 / 动作数量	$\mathcal{A}$ 是离散动作集合, $ \mathcal{A} $ 是其中的动作数。
$Q(s,a;\theta)$	Online Action-value Function	在线动作价值函数	参数为 θ 的在线 Q 网络对 (s,a) 的估计。
$Q(s,a;\theta^-)$	Target Action-value Function	目标动作价值函数	参数为 θ^- 的目标 Q 网络, 用于构造较稳定的 TD target。
θ / θ^-	Online / Target Parameters	在线 / 目标网络参数	θ 每次训练更新; θ^- 按较慢频率从 θ 同步。
γ	Discount Factor	折扣因子	控制下一状态价值在 target 中的权重。
α	Learning Rate / Step Size	学习率 / 步长	表格 Q-learning 中吸收 TD error 的比例。
ϵ	Exploration Probability	探索概率	epsilon-greedy 中随机探索的概率。
y	TD Target	TD 目标	当前 Q 预测需要逼近的监督信号。
δ / δ_i	TD Error	TD 误差	target 与当前 Q 预测之差; δ_i 表示第 i 条样本的误差。
$L(\theta)$	Loss Function	损失函数	训练在线 Q 网络时最小化的回归损失。
$V(s)$	State-value Stream	状态价值分支	Dueling DQN 中表示“状态整体有多好”的标量输出。
$A(s,a)$	Advantage Stream	动作优势分支	Dueling DQN 中表示动作相对该状态共同基线好多少的输出; 这里的斜体 A 不是动作集合 $\mathcal{A}$ 。
p_i / $P(i)$	Priority / Sampling Probability	优先级 / 采样概率	PER 中第 i 条样本的优先级及其被抽到的概率。

符号/缩写	English	中文	含义
α_p	Priority Exponent	优先级指数	控制 PER 采样对优先级的偏向程度；与学习率 α 不同。
β_{IS}	IS Correction Exponent	重要性采样修正指数	控制 PER 对非均匀采样偏差的修正强度。
ε_p	Priority Floor	优先级下限常数	防止零 TD error 样本永远无法被采样的小正数；与探索率 ε 不同。
w_i	Importance Sampling Weight	重要性采样权重	PER 中修正非均匀采样偏差的样本权重。
N	Replay-buffer Size	回放池样本数	PER 权重公式中的样本总数。
TD	Temporal Difference	时序差分	用即时奖励和下一状态估计构造学习目标。
PER	Prioritized Experience Replay	优先经验回放	按学习价值而非完全均匀地抽取 replay 样本。
SGD	Stochastic Gradient Descent	随机梯度下降	用随机 mini-batch 更新神经网络参数的方法。
IS	Importance Sampling	重要性采样	用权重修正采样分布变化的方法。

6.2 本章目标

学完本章，你应该能沿着一条完整因果链解释：

1. 为什么表格 Q-learning 不能直接处理高维状态。
2. 为什么“把 Q 表换成神经网络”仍然不够稳定。
3. Experience Replay 和 Target Network 分别解决什么问题。
4. Double DQN、Dueling DQN、PER 各自针对哪个剩余缺陷。
5. 哪些改进改变 target，哪些改变网络结构，哪些改变采样分布。

6.3 本章主线：每个组件都必须对应一个问题

先看整章的问题树：

表格 Q-learning 无法处理巨大状态空间

- > 用神经网络近似 $Q(s, a)$
- > 新问题 1: 连续交互样本高度相关
 - > Experience Replay
- > 新问题 2: 预测网络和训练 target 同时变化
 - > Target Network
- > 得到基础 DQN

基础 DQN 仍有三个典型缺陷

- > max 容易放大 Q 估计噪声
 - > Double DQN
- > 每个动作 Q 值包含大量重复的状态价值信息
 - > Dueling DQN
- > 均匀 replay 不区分样本学习价值
 - > Prioritized Experience Replay

用一张表先固定它们的边界：

方法	它解决的问题	它主要改变什么
Experience Replay	连续样本相关、单条经验只用一次	训练数据的组织方式
Target Network	TD target 随在线网络同步移动	target 的计算网络
Double DQN	max 操作放大过估计	target 中“选动作”和“估值”的分工
Dueling DQN	状态共同价值被每个动作输出重复学习	Q 网络的输出结构
PER	均匀采样浪费在已学好的样本上	replay buffer 的采样分布

这张表是本章的脉络。后面每个公式都从对应问题出发。

6.4 从 Q-learning 推到 DQN

6.4.1 起点：表格 Q-learning

表格 Q-learning 对一个格子做增量更新：

$$Q(s,a) \leftarrow Q(s,a) + \alpha[y - Q(s,a)]$$

其中 target 是：

$$y = r + \gamma(1-d)\max_a Q(s',a')$$

式中：

- r 是这一步真实获得的 reward。
- $\max_a Q(s',a')$ 是下一状态可达到的最大估计价值。
- $1-d$ 是终止掩码；终止时 $d=1$ ，未来价值自动变成 0。
- $y - Q(s,a)$ 就是 TD error。

表格方法的问题不是更新逻辑错误，而是无法存下所有状态。例如输入是一张游戏画面时，几乎不可能为每种像素组合建立一个 Q 表格行。

6.4.2 第一步：用神经网络替代 Q 表

DQN 用参数为 θ 的神经网络近似 Q 函数：

$$Q(s, a; \theta)$$

在离散动作空间中，网络通常只输入状态 s ，一次输出所有动作的 Q 值：

```
state s
  -> Q network
  -> [Q(s,a1), Q(s,a2), ..., Q(s,an)]
```

选择动作时：

- 以 ϵ 概率随机探索。
- 以 $1-\epsilon$ 概率选择 Q 值最大的动作。

6.4.3 第二步：把表格更新改写成回归

神经网络不能只修改某一个表格格子。它能做的是：让当前预测 $Q(s, a; \theta)$ 靠近 target y 。

因此把 Q-learning 改写为监督学习式回归：

```
input:      (s, a)
target:     y
prediction: Q(s, a; theta)
error:      y - Q(s, a; theta)
```

平方损失可以写成：

$$L(\theta) = \mathbb{E}[(y - Q(s, a; \theta))^2]$$

实际 DQN 常用 Huber loss：小误差区域近似平方损失，大误差区域近似绝对值损失，从而降低异常 TD error 对梯度的冲击。

6.4.4 为什么“直接用神经网络在线训练”仍不稳定

如果每和环境交互一步，就立刻用最新 transition 训练同一个网络，会遇到两个核心问题。

6.4.4.1 问题一：样本高度相关

连续三帧游戏画面通常只发生很小变化：

```
s_t      : 小车在中央
s_{t+1} : 小车略微向右
s_{t+2} : 小车继续向右
```

按时间顺序直接训练，连续 mini-batch 会被局部轨迹主导，不符合 SGD 希望数据较充分混合的条件。

6.4.4.2 问题二：target 一直移动

如果预测和 target 都使用当前网络：

$$y = r + \gamma(1-d)\max_a Q(s', a'; \theta)$$

每次更新 θ 时：

- 当前预测 $Q(s,a;\theta)$ 变了。
- 用来监督它的 target y 也立刻变了。

这像一边追目标，一边同步移动目标，容易产生震荡甚至发散。

基础 DQN 的两个核心组件正好分别处理这两个问题。

6.5 基础 DQN：Replay Buffer 加 Target Network

6.5.1 Experience Replay：先解决数据相关性

Replay buffer 保存历史 transition：

(s, a, r, s', d)

字段含义：

字段	含义
s	执行动作前的当前状态。
a	agent 实际执行的动作。
r	执行动作后得到的即时奖励。
s'	环境到达的下一状态。
d / done	是否已经终止；1 表示没有后续价值，0 表示仍可从 s' 估计未来价值。 <code>done</code> 只是常见代码字段名。

训练时不只使用刚产生的 transition，而是从 replay buffer 随机抽取 mini-batch。

它带来两个作用：

1. **打散相关性**：同一个 batch 可以混合不同时间、不同 episode 的经验。
2. **复用经验**：一条环境交互数据可以被训练多次，提高样本效率。

它没有解决 target 移动问题，所以还需要 target network。

6.5.2 Target Network：再解决移动目标

DQN 维护两个结构相同的 Q 网络：

网络	参数	职责	更新方式
Online Network	θ	产生当前 Q 预测并接受梯度更新	每个训练 step 更新
Target Network	θ'	只负责计算 TD target	每隔固定步数从 θ 同步

target 改成：

$$y = r + \gamma(1-d)\max_a Q(s', a'; \theta')$$

在两次同步之间， θ' 保持不变，所以在线网络面对的是相对稳定的监督目标。

注意：Target Network 不是第二个独立学习目标。它只是在线网络的滞后副本，用时间上的延迟换取 target 稳定性。

6.5.3 完整 DQN 训练流程

```

initialize online network Q(theta)
initialize target network Q(theta-) <- Q(theta)
initialize replay buffer

repeat:
    1. use epsilon-greedy to choose action a
    2. execute a and observe (r, s', d)
    3. store (s, a, r, s', d) in replay buffer

    4. sample a random minibatch from replay buffer
    5. for each transition:
        y = r + gamma * (1-d) * max_a' Q(s', a'; theta-)
    6. update theta to reduce Huber(y - Q(s, a; theta))

    7. periodically copy theta to theta-
    8. move to s'

```

这就是基础 DQN。不要把它记成零散技巧，应该记成：

```

Q-learning target
+ neural Q function
+ replay buffer
+ target network
= DQN

```

6.5.4 数值例子：target、TD error 和 loss 怎么算

假设一条非终止 transition 中：

```

r = 1
gamma = 0.9
d = 0
target network 在 s' 的输出 = [2, 4]
online network 当前 Q(s,a) = 3

```

先计算下一状态最大价值：

$$\max_a Q(s', a'; \theta) = 4$$

再计算 target：

$$y = 1 + 0.9 \times 4 = 4.6$$

TD error：

$$\delta = 4.6 - 3 = 1.6$$

平方误差：

$$\delta^2 = 2.56$$

如果同一条 transition 是终止转移, 即 $d=1$:

$$y=1+0.9\times(1-1)\times 4=1$$

这就是 `done` 字段存在的全部意义: 告诉 target 终止之后不能再凭空接入未来价值。理解 DQN 本身不需要引入任何特定环境库。

6.5.5 基础 DQN 解决了什么, 还没解决什么

基础 DQN 已经解决:

- 高维状态下无法维护 Q 表。
- 连续样本高度相关。
- bootstrap target 同步快速移动。

但它仍然没有解决:

- max 操作带来的 Q 值过估计。
- 网络对“状态共同价值”和“动作相对差异”的学习效率。
- replay buffer 中不同样本的学习价值不同。

下面三个改进依次对应这三个问题。

6.6 Double DQN: 修正 max 带来的过估计

6.6.1 问题: max 为什么容易高估

假设每个动作的估计都等于:

$$\text{estimated } Q = \text{true } Q + \text{estimation noise}$$

即使每个动作的噪声平均为 0, 取最大值时也更容易选中“恰好被正噪声抬高”的动作。

基础 DQN 的 target network 同时完成两件事:

1. 从下一状态选择 Q 最大的动作。
2. 用这个最大 Q 值评价该动作。

同一份噪声既影响选择, 又影响估值, 因此正噪声容易被 max 放大。

6.6.2 思路: 选动作和估值分开

Double DQN 让在线网络负责选择动作:

$$a^* = \arg \max_a Q(s', a'; \theta)$$

再让目标网络只评价这个动作:

$$y = r + \gamma(1-d)Q(s', a^*; \theta')$$

一句话:

```
online network:  decide which action
target network:  evaluate that action
```

6.6.3 数值例子

假设在线网络和目标网络对下一状态给出：

网络	动作 1	动作 2
Online Network	9.9	10.4
Target Network	10.1	10.0

基础 DQN 只看 target network 的最大值：

$$\max(10.1, 10.0) = 10.1$$

Double DQN 先由 online network 选择动作 2，再由 target network 评价动作 2：

$$Q(s', a_2; \theta) = 10.0$$

这个例子中，Double DQN 没有重复选择 target network 中被噪声抬高的动作 1。

注意：Double DQN 不保证每一个样本的 target 都更小。它降低的是“同一估计噪声同时负责选择和估值”造成的系统性过估计。

6.6.4 Double DQN 改了什么

它只修改 target 的计算方式：

- replay buffer 不变。
- online/target 两个网络仍然存在。
- Q 网络结构可以完全不变。
- epsilon-greedy 行为策略不变。

因此 Double DQN 可以继续与 Dueling DQN、PER 组合。

6.7 Dueling DQN：拆开状态共同价值与动作相对差异

6.7.1 先看它要解决的问题

假设某个状态下有三个动作，Q 值分别是：

[5.1, 5.0, 4.9]

这里包含两类信息：

1. **共同信息**：这个状态整体价值大约是 5。
2. **差异信息**：动作之间只有 +0.1、0、-0.1 的相对差异。

普通 DQN 直接输出三个 Q 值，相当于让三个动作输出分别学习那一大块重复的“状态整体价值”。

在很多状态中，动作差异暂时并不重要。例如小车和障碍物还很远时，短期内向左或向右可能差别很小；但“当前局面整体安全还是危险”仍然值得先学准。

Dueling DQN 的动机就是：

让网络单独学习状态的共同价值，再单独学习动作之间的相对差异。

6.7.2 两个分支分别表示什么

Dueling DQN 把网络输出拆成两个分支：

- $V(s)$: **State-value Stream** (状态价值分支), 输出一个标量, 表示状态 s 的共同价值基线。
- $A(s,a)$: **Advantage Stream** (动作优势分支), 为每个动作输出一个分数, 表示动作相对共同基线的差异。

这里必须区分两个符号：

- 斜体 $A(s,a)$: 某个动作的 advantage 输出。
- 花体 $\mathcal{A}$: 全部离散动作组成的 action set。

6.7.3 网络结构发生了什么变化

```

state -> shared encoder
                        -> value head V(s): 1 scalar
                        -> advantage head A(s,.): |A| scores

V stream + centered A stream -> Q(s,.)

```

其中：

- shared encoder 是两条分支共用的状态特征提取器。
- value head 是输出 $V(s)$ 的分支。
- advantage head 是输出每个动作 $A(s,a)$ 的分支。
- 最后的 aggregation 层把两条分支重新合成为每个动作的 Q 值。

Dueling DQN 仍然输出 Q 值, 最终选动作的方式没有改变。

6.7.4 为什么不能直接写成 (Q=V+A)

最直觉的写法是：

$$Q(s,a) = V(s) + A(s,a)$$

但这个分解不唯一。任意给定常数 c ：

$$[V(s)+c] + [A(s,a)-c] = V(s) + A(s,a)$$

也就是说：

- V 整体加 10。
- 所有动作的 A 整体减 10。
- 最终 Q 值完全不变。

网络只看最终 Q loss, 无法判断这 10 应该属于 V 还是 A 。这就是不可辨识性：同一组 Q 值对应无数组 V/A 分解。

6.7.5 正确做法：把 advantage 中心化

假设离散动作共有 $n=|\mathcal{A}|$ 个。先把 advantage 分支的平均输出单独记为 $m_A(s)$ ：

$$m_A(s) = [A(s,a_1) + A(s,a_2) + \dots + A(s,a_n)]/n$$

再用这个均值把 raw advantage 中心化：

$$Q(s,a)=V(s)+A(s,a)-m_A(s)$$

其中 $a_1, \dots, a_n$ 是动作集合中的全部动作。这个写法与“减去所有动作 advantage 的均值”完全等价，但更容易直接看出每一项在做什么：

```
当前动作的 raw advantage
- 所有动作 raw advantage 的平均值
= centered advantage
```

中心化后，所有动作 advantage 的平均值为 0，因此：

$$[Q(s,a_1)+Q(s,a_2)+\dots+Q(s,a_n)]/n=V(s)$$

这时 $V(s)$ 有了明确含义：它等于该状态下所有动作 Q 值的平均基线；每个动作的中心化 advantage 决定它在基线上方还是下方。

6.7.6 数值例子：完整算一遍

假设一个状态有三个动作，网络两条分支输出：

```
V(s) = 5
raw A(s,.) = [2, 1, 0]
```

第一步，计算 raw advantage 的平均值：

$$(2+1+0)/3=1$$

第二步，中心化 advantage：

```
[2, 1, 0] - 1 = [1, 0, -1]
```

第三步，与状态基线相加：

```
Q(s,.) = 5 + [1, 0, -1]
        = [6, 5, 4]
```

结果的含义：

- 状态整体基线是 5。
- 动作 1 比基线好 1，所以 Q 值是 6。
- 动作 2 与基线相同，所以 Q 值是 5。
- 动作 3 比基线差 1，所以 Q 值是 4。

这比一上来背 $Q=V+A$ 更重要：**Dueling** 的重点不是公式拆分，而是把重复的状态信息和动作差异信息交给不同分支学习。

6.7.7 Dueling DQN 改了什么，没有改什么

项目	是否改变	说明
Q 网络内部结构	是	单一输出头改成 value head 和 advantage head。
最终输出	否	仍然输出每个离散动作的 Q 值。
DQN Bellman target	否	仍可使用基础 DQN target。
Double DQN target	可组合	可以用 Dueling 网络，同时使用 Double DQN 的 target。
Replay Buffer	否	数据存储和采样逻辑不因 Dueling 改变。
行为策略	否	仍可使用 epsilon-greedy。
过估计问题	不直接解决	过估计主要由 Double DQN 处理。

所以要牢牢记住：

Double DQN 改 target；Dueling DQN 改网络结构。两者不是竞争关系，可以同时使用。

6.7.8 什么时候更可能有帮助

Dueling DQN 更适合：

- 离散动作空间。
- 很多状态下动作差异较小，但状态本身好坏很明显。
- 不同动作共享大量状态特征的任务。

它不是任何任务都必然提升，也不负责解决连续动作输出、探索不足或 replay 分布偏差。

6.7.9 面试回答模板

Dueling DQN 解决的是 Q 表示学习效率问题。普通 DQN 直接输出每个动作的 Q 值，而 Dueling 把网络拆成状态价值分支 $V(s)$ 和动作优势分支 $A(s,a)$ 。由于直接写 $Q=V+A$ 不可辨识，实际会减去所有动作 advantage 的均值再聚合。它只改变网络结构，不改变 replay 和 Bellman target，因此常与 Double DQN、PER 一起使用。

6.8 PER：让更有学习价值的 transition 更常被采样

6.8.1 问题：均匀 replay 一视同仁

普通 replay buffer 对所有 transition 均匀采样。

但不同样本的学习价值不同：

- TD error 很小：当前网络已经能较好预测。
- TD error 很大：当前预测与 target 差距大，通常更值得再次学习。

PER 的思路是让 TD error 大的样本更常被抽到。

6.8.2 从 TD error 构造优先级

第 i 条 transition 的 TD error:

$$\delta_i = y_i - Q(s_i, a_i; \theta)$$

优先级:

$$p_i = |\delta_i| + \epsilon_p$$

$\epsilon_p > 0$ 防止 TD error 暂时为 0 的样本永远失去采样机会。

采样概率:

$$z_p = p_1^{\alpha_p} + p_2^{\alpha_p} + \dots + p_N^{\alpha_p}$$

$$P(i) = p_i^{\alpha_p} / z_p$$

z_p 是归一化常数, 即 replay buffer 中所有样本优先级幂次之和。

α_p 的作用:

- $\alpha_p = 0$: 所有 $p_i^0 = 1$, 退化为均匀采样。
- α_p 越大: 越偏向高优先级样本。

6.8.3 为什么还需要重要性采样修正

非均匀采样改变了训练数据分布。高 TD error 样本被过度代表, 直接训练会引入 sampling bias。

PER 给每条被采样的数据乘 importance sampling weight:

$$w_i = [N P(i)]^{-\beta_{IS}}$$

直觉:

- $P(i)$ 大的样本经常被抽到, 因此单次更新权重应更小。
- $P(i)$ 小的样本很少被抽到, 因此抽到时权重应更大。

β_{IS} 控制修正强度:

- $\beta_{IS} = 0$: 不修正。
- $\beta_{IS} = 1$: 完整使用该重要性权重形式。

实现中常把 batch 内 w_i 除以最大权重, 避免整体梯度尺度突然变大。

6.8.4 数值直觉

假设两条样本:

```
sample 1: |TD error| = 0.2
sample 2: |TD error| = 1.8
```

忽略很小的 ϵ_p 时, 第二条样本优先级远高于第一条, 因此会更常被抽到。但因为第二条样本被重复看到的概率更高, 它的 loss 还要乘较小的 w_i , 防止训练分布被它完全主导。

6.8.5 PER 的完整循环

1. new transition enters replay buffer
2. assign an initial priority
3. sample minibatch according to $P(i)$
4. compute weighted TD loss using w_i
5. update online network
6. recompute sampled transitions' TD errors
7. write new priorities back to replay buffer

PER 改的是采样分布，不改变 DQN 或 Double DQN 的 target 结构。

6.9 Rainbow DQN：这些改进如何组合

Double、Dueling、PER 解决的是不同问题，所以可以叠加。Rainbow DQN 的核心思想就是组合多种互补改进。

组件	一句话定义	主要针对的问题
Double DQN	online 选动作，target 估值	Q 值过估计
Dueling DQN	状态价值和动作优势分支	Q 表示学习效率
PER	按 TD error 调整 replay 概率	均匀采样效率
Multi-step Learning（多步学习）	用连续多步真实 reward 构造 target	奖励传播过慢
Distributional RL（分布强化学习）	预测 return 的概率分布，而不只预测均值	价值分布表达不足
NoisyNet（噪声网络）	在网络参数中加入可学习噪声进行探索	epsilon-greedy 探索过于简单

本章面试重点不是背 Rainbow 的所有工程细节，而是能说明：

这些组件没有重复做同一件事；它们分别修改 target、网络结构、采样方式、回报形式、价值表示和探索机制。

6.10 DQN 的适用范围与完整例子

6.10.1 适用范围

DQN 适合：

- 离散动作空间。
- 可以一次输出并比较所有动作 Q 值。
- 状态维度较高，需要函数逼近。

DQN 不适合：

- 高维连续动作空间，因为无法枚举所有动作取 max。
- 极端稀疏奖励且简单 epsilon-greedy 很难探索到有效行为的任务。
- 环境数据分布变化极端、普通 replay 无法覆盖关键状态的任务。

6.10.2 CartPole 例子：从建模到训练

CartPole（小车倒立摆）是经典离散控制任务。目标是左右推动小车，让杆保持直立。

状态：

状态维度	含义
cart position	小车位置
cart velocity	小车速度
pole angle	杆角度
pole angular velocity	杆角速度

动作：

```
0: push left
1: push right
```

DQN 网络：

```
input: 4-dimensional state
output: [Q(s,left), Q(s,right)]
```

一轮交互和训练：

```
1. epsilon-greedy chooses left or right
2. environment returns reward, next state, and done
3. transition enters replay buffer
4. random minibatch is sampled
5. target network computes y
6. online network minimizes TD loss
7. target network is periodically synchronized
```

如果基础 DQN 有明显过估计，可以加入 Double DQN；如果许多状态下左右动作差异小，可以尝试 Dueling DQN；如果少数失败或关键转移很重要，可以尝试 PER。

这最后一句就是算法选型逻辑：先观察具体缺陷，再选择对应改进。

6.11 高频对比与面试问题

6.11.1 Replay Buffer 和 Target Network 分别解决什么

- Replay Buffer 处理训练数据问题：打散连续样本相关性并复用历史经验。
- Target Network 处理监督目标问题：让 bootstrap target 在一段时间内保持稳定。

6.11.2 Double DQN 和 Dueling DQN 有什么区别

- Double DQN 修改 target 计算，缓解 max 导致的过估计。
- Dueling DQN 修改网络结构，拆开状态共同价值与动作相对差异。
- 两者可以同时使用。

6.11.3 Dueling 为什么要减去 advantage 均值

直接写 $Q=V+A$ 时， V 加任意常数、所有 A 减同一常数都不会改变 Q ，因此分解不唯一。减去动作 advantage 的均值后，中心化 advantage 平均为 0， $V(s)$ 就对应所有动作 Q 值的平均基线。

6.11.4 PER 为什么需要 IS weight

PER 非均匀采样会改变训练分布。IS weight 降低高概率样本的单次影响、提高低概率样本被抽到时的影响，用于修正 sampling bias。

6.11.5 DQN 为什么不适合连续动作

DQN 需要比较所有动作并计算 $\max_a Q(s,a)$ 。连续动作有无限多个候选，无法像离散动作一样直接枚举。

6.12 本章练习

1. 从表格 Q-learning 更新推导 DQN target 和 loss。
2. 分别解释 Replay Buffer 与 Target Network 解决的问题。
3. 已知 $r=2$ 、 $\gamma=0.9$ 、 $d=0$ 、target network 在 s' 输出 $[3,5]$ ，计算 DQN target。
4. 解释 Double DQN 为什么把动作选择和动作估值拆开。
5. 某 Dueling 网络输出 $V(s)=4$ 、raw $A(s,\cdot)=[3,1,2]$ ，计算三个动作的 Q 值。
6. 说明 Double DQN 与 Dueling DQN 为什么可以组合。
7. 解释 PER 中 α_p 、 β_{IS} 和 ϵ_p 的作用。

▼ 参考答案 (点击展开)

1. **从 Q-learning 到 DQN**：表格更新是 $Q(s,a) \leftarrow Q(s,a) + \alpha[y - Q(s,a)]$ ，其中 $y = r + \gamma(1-d)\max_{a'} Q(s',a')$ 。用神经网络 $Q(s,a;\theta)$ 替代表格后，把 y 当作固定监督目标，最小化 $L(\theta) = \mathbb{E}[(y - Q(s,a;\theta))^2]$ 或 Huber loss。DQN 使用 target network 计算 $y = r + \gamma(1-d)\max_{a'} Q(s',a';\theta^-)$ 。
2. **Replay 与 Target Network**：Replay Buffer 随机混合不同时间的 transition，降低连续样本相关性并复用经验；Target Network 是在线网络的滞后副本，让计算 target 的参数 θ 在一段时间内保持不变。前者处理数据分布，后者处理移动目标。
3. **DQN target**：下一状态最大 Q 值是 5，因此 $y = 2 + 0.9 \times (1-0) \times 5 = 6.5$ 。如果 $d=1$ ，则 $y=2$ 。
4. **Double DQN**：对多个带噪声 Q 估计取 max 时，容易选中被正噪声高估的动作。基础 DQN 用同一个 target network 同时选动作和估值；Double DQN 用 online network 选择 a ，再用 target network 评价 $Q(s',a;\theta^-)$ ，降低同一噪声被选择和估值重复放大的程度。
5. **Dueling 数值题**：raw advantage 均值为 $(3+1+2)/3=2$ ，中心化后是 $[1,-1,0]$ 。加上 $V(s)=4$ ，得到 $Q(s,\cdot)=[5,3,4]$ 。
6. **为什么可以组合**：Double DQN 改的是 Bellman target 中动作选择与动作估值的分工；Dueling DQN 改的是 Q 网络内部的 value/advantage 输出结构。二者作用位置不同，因此可以用 Dueling 网络作为 online/target network，同时使用 Double DQN target。
7. **PER 参数**： α_p 控制采样对优先级的偏向，0 表示均匀采样； β_{IS} 控制重要性采样偏差修正强度； ϵ_p 给所有样本保留正优先级，避免 TD error 为 0 的样本永远无法再次被采样。